

Vamos conhecer o Gold Miners

Guia de estudo e experimentação · Marco Aurélio Santos de Souza
Projeto individual de Sistemas Multiagentes · UFSC · Prof. Jomi F. Hübner · 2026.2

Uma primeira conversa sobre o projeto

Você não precisa conhecer a história deste trabalho para começar. Imagine um pequeno grupo procurando ouro e levando cada peça até um depósito. Há um mapa, obstáculos e outros mineradores. Cada agente precisa escolher o que fazer e acompanhar o resultado de suas ações.

O interesse está nas decisões: continuar explorando ou recolher uma peça já conhecida? Dividir uma descoberta com os colegas? Desistir de uma rota que deixou de funcionar? Essas perguntas aproximam o jogo dos problemas de coordenação que encontramos em sistemas de software.

O que você consegue experimentar agora

Esta primeira versão executa o tutorial JaCaMo, com exploração, coleta e depósito. Inclui uma verificação pequena e controlada, além de uma sessão com quatro mineradores. A competição entre duas equipes, a coordenação por reservas e a organização Moise são próximas etapas, descritas no plano de evolução.

O código inicial vem do tutorial oficial. João Leite adaptou o exemplo a partir do trabalho de Rafael Bordini, Jomi Hübner e Maicon Zatelli. Os créditos continuam preservados. As contribuições desta preparação estão nas correções documentadas, na execução controlada, nas verificações e no desenho de investigação.

Por onde seguir

Leia primeiro este guia. Depois, abra o [código explicado linha por linha](#). Você encontrará cada arquivo do núcleo de execução com o trecho original ao lado da explicação em português. As linhas mantêm a numeração do arquivo para facilitar a navegação no editor.

O apêndice cobre agentes, ações internas, ambiente, visualização, testes, lançador e configurações. Comentários originais e linhas vazias podem ser ocultados na tela. Bibliotecas, arquivos gerados e o Gradle Wrapper não são tratados como código escrito para o projeto.



Abrir laboratório no GitHub

github.com/MASS-Marco/gold-miners-jacamo

Preparar e executar com calma

No navegador

Abra o repositório e use o botão **Abrir no GitHub Codespaces**. Esse recurso usa sua conta GitHub e prepara um ambiente de desenvolvimento conforme a configuração do projeto. Depois da preparação, abra o terminal e execute:

```
python3 scripts/laboratorio.py validar
python3 scripts/laboratorio.py executar --modo coletar --cenario 3 --segundos 15
```

A configuração de contêiner está incluída. As verificações documentadas foram executadas em Windows; a execução no Codespaces ainda não foi conferida.

No computador

Você pode usar **Code** → **Download ZIP** ou clonar o repositório. Prepare Java 21 e Python 3.10 ou superior. Na pasta do projeto, o comando Windows é:

```
.\laboratorio.cmd validar
```

No acervo local de Marco, o lançador encontra automaticamente o Java isolado e o Python já preparados para a disciplina. Em outra máquina, ele usa `JAVA_HOME` ou o Java disponível no sistema. O Gradle e as dependências poderão ser baixados na primeira utilização.

O que observar

Ao final da verificação, procure `VERIFICACAO_GOLD_MINERS_APROVADA`. Essa mensagem só aparece depois de conferir regras do modelo e uma entrega executada por um agente. Se houver falha, abra a pasta de registros indicada pelo comando.

Na sessão de estudo, o processo termina após o tempo solicitado. Isso é esperado. Para ver o mapa em uma máquina com interface gráfica, acrescente `--interface`. A execução gráfica foi preservada para estudo, mas não fez parte da conferência desta preparação.

Entender a linguagem dos agentes

AgentSpeak representa crenças, objetivos e planos. Pense numa crença como uma informação que o agente possui naquele momento. Ela pode vir da percepção ou de seu próprio raciocínio, e pode ficar desatualizada.

Escrita	Como ler no exemplo
<code>free.</code>	O agente começa acreditando que está disponível.
<code>!pos(X,Y)</code>	Tentar alcançar exatamente a posição X,Y. O ponto de exclamação introduz um objetivo.
<code>+!pos(X,Y)</code>	Um plano pode reagir à adoção desse objetivo.
<code>: pos(X,Y)</code>	Contexto do plano: essa informação precisa ser satisfeita para selecionar a alternativa.
<code><-</code>	Começo do corpo do plano, com as ações e os subobjetivos.
<code>?carrying_gold</code>	Conferir na base de crenças se o agente está carregando ouro.
<code>-+last_dir(D)</code>	Substituir a crença anterior sobre a última direção.
<code>!!handle(Gold)</code>	Criar uma intenção separada para tratar a coleta.

Os termos X e Y começam com maiúscula e funcionam como variáveis. Palavras como `free`, `gold` e `pick` são símbolos do programa. O nome ajuda a leitura, mas quem define seu significado operacional são os planos e as operações correspondentes.

Um detalhe que faz diferença

`pos(X,Y)` é uma crença sobre a posição atual. `!pos(X,Y)` é um objetivo de chegar lá. A mesma escrita básica aparece nos dois casos, mas o prefixo muda o papel da expressão.

No agente de coleta, planos marcados com `atomic` evitam intercalar outras intenções daquele agente durante uma preparação curta. Isso não transforma automaticamente toda a coleta numa operação indivisível entre vários agentes. A reserva compartilhada de um alvo exigirá uma operação própria no artefato.

Acompanhar uma entrega de ponta a ponta

Vamos seguir a verificação incluída no laboratório. O agente de teste começa em 1,0, percebe ouro em 1,1 e segue um percurso conhecido. Ele desce, coleta, sobe, vai à esquerda e deposita em 0,0. O teste é pequeno de propósito: fica fácil reconhecer onde cada responsabilidade aparece.

Arquivo	O que procurar
<code>configuracoes/verificacao.jcm</code>	O agente de teste, seu artefato de percepção e o observador de confirmação.
<code>src/test/agt/verificacao.asl</code>	A sequência de ações e as esperas por posição e carga.
<code>MiningPlanet.java</code>	Operações <code>down</code> , <code>pick</code> e <code>drop</code> , seguidas da atualização das percepções.
<code>WorldModel.java</code>	Retirada do ouro do mapa, registro da carga e incremento do total depositado.
<code>ValidationProbe.java</code>	Conferência de um depósito, carga vazia e conservação do recurso.

Por que conferir o mundo

Uma mensagem como “vou coletar” mostra a intenção. Uma mensagem de conclusão do agente mostra o que ele relata. A verificação procura também a alteração correspondente no modelo do mundo. Esse cuidado será importante quando houver atrasos, tentativas repetidas e mais de uma equipe.

O método `move` da base retorna `true` mesmo quando não muda a posição por encontrar uma célula ocupada. Por isso, a verificação observa a posição atingida. Um valor de retorno isolado não substitui o entendimento do método.

Na coleta do tutorial, `get_direction` consulta o modelo global. Isso ajuda o exemplo a encontrar caminhos, mas deve mudar antes de afirmar que as estratégias competem com informação local. O observador de teste pode inspecionar o mundo inteiro; a política do agente deve respeitar a informação permitida.

Três exercícios para aprender fazendo

1 Explorar é diferente de coletar

Execute uma sessão com `--modo explorar --cenario 1` e outra com `--modo coletar --cenario 3`. Abra os arquivos `miner.asl` e `miner-coleta.asl`. Tente localizar onde a segunda versão reage à descoberta de ouro e adota a tarefa de entrega.

Pista para conferir: a exploração escolhe destinos para visitar. A coleta acrescenta memória de ouro conhecido, tratamento do alvo, verificação de carga e retorno ao depósito. Mover-se bastante não demonstra que o objetivo coletivo foi cumprido.

2 O depósito mudou de lugar

Encontre as referências a `pos(0,0)` no agente de coleta. O ambiente também fornece a crença `depot(_,X,Y)`. Como o plano poderia buscar essas coordenadas em vez de supor sempre 0,0?

Pista para conferir: a localização deve ser consultada antes de escolher o destino e antes de testar a condição de depósito. Depois, crie uma verificação com depósito deslocado. O lançador atual restringe a coleta aos cenários 1–3; sua ampliação deve acompanhar a correção do agente e o teste correspondente.

3 Duas pessoas querem a mesma tarefa

Imagine dois agentes tentando reservar o mesmo recurso ao mesmo tempo. Desenhe o estado do quadro antes e depois das duas solicitações. Qual deles recebeu a reserva? O que o outro observa? Quando a reserva pode voltar a ficar disponível?

Pista para conferir: conceder a reserva deve ser uma operação indivisível. Uma versão da tentativa ajuda a rejeitar mensagens atrasadas. Se o ouro já foi coletado, a redistribuição precisa considerar a carga real.

Registrar o aprendizado

Depois de cada exercício, escreva três frases: o que esperava, o que aconteceu e o que verificou para chegar à conclusão. Guarde a configuração e os registros. Antes de salvar uma etapa no Git, examine `git diff` e repita a verificação afetada pela mudança.

Da prática à organização e aos experimentos

O próximo núcleo será uma competição entre duas equipes. Primeiro virá a coleta independente, depois a coordenação com um quadro de reservas. A organização Moise será uma etapa posterior, mantendo os critérios de decisão comparáveis.

O modelo BPMN disponível em [docs/modelos](#) representa uma tentativa, a conferência do resultado e a reavaliação. Ele é um modelo de análise, sem execução em motor de processos. O contrato de missões é uma proposta; ainda não descreve eventos emitidos pelo tutorial.

Uma pergunta para levar ao experimento

Se a coordenação evitar três viagens repetidas, mas obrigar todos a esperar muito para reservar um alvo, ela ainda compensa? Para responder, precisamos medir o que foi entregue e o esforço consumido. Vitória, quantidade coletada, valor depositado e tempo de processamento respondem perguntas diferentes.

O protocolo em `experimentos/protocolo.json` propõe uma campanha futura. Não contém resultados de partidas já realizadas. As evidências atuais se limitam às verificações funcionais e à sessão de estudo.

Eficiência eficácia e efetividade

Eficiência relaciona valor entregue e ações consumidas. Eficácia verifica o cumprimento da meta no prazo. Efetividade simulada investiga a melhora útil em relação a uma referência comparável, inclusive sob perturbações. Robustez ajuda a sustentar esse resultado, mas não substitui a avaliação do efeito obtido. Impacto em transporte ou homologação exigiria outra investigação.

Execute `python scripts/indicadores.py experimentos/exemplo-indicadores.json` para acompanhar os cálculos. Os números são didáticos e permanecem identificados como `ilustrativo`. O módulo recebe registros; a coleta automática das métricas do simulador ainda será implementada. As fórmulas e os casos de erro são conferidos por `python scripts/verificar_indicadores.py`.

Quando algo não funcionar

Se o Java não for encontrado, confira a versão e `JAVA_HOME` no terminal utilizado. Se a compilação falhar ao obter dependências, preserve a mensagem e verifique o acesso à fonte indicada. Se um agente ficar parado, procure sua posição, intenção ativa e último efeito observado, antes de alterar a estratégia.

A ação interna `random_direction` é uma alternativa antiga preservada no pacote e não é usada pelos mineradores executados aqui. Sua lógica de coordenadas precisa de revisão antes de adotá-la. Esse é um exemplo de por que estar presente no projeto não significa já ter sido validado em todas as situações.

Referências para continuar

Tutorial Gold Miners · MAPC 2006 · Documentação ORA4MAS · OMG BPMN · Cossentino, Lopes e Sabatucci, 2020.

Essas fontes têm funções diferentes: tutorial de programação, regras históricas, documentação de plataforma, especificação de notação e artigo de pesquisa. A relação BPMN–Moise possui antecedentes; o interesse do projeto é estudar sua aplicação e seus limites neste ambiente.